

CRUZR S2 · VLA · TELEOPERACIÓN

Cruzer S2 v0.2.0: teleoperación, captura de datos y evolución del VLA

Compatibilidad v0.2.0, PICO 4 Ultra Enterprise, captura de demostraciones, diseño de datasets y estrategia de entrenamiento.

Versión documental 1.1
21 de agosto de 2026

Contenido

1. Propósito y alcance

Convenciones de evidencia

2. Estado de referencia del sistema

2.1 Robot real

2.2 Interfaces observables relevantes

3. Qué hace actualmente el VLA suministrado

3.1 Entrada, salida y tareas

3.2 Dataset original inspeccionado

3.3 Resultado shadow

4. VLA frente a programación tradicional

5. Matriz de compatibilidad

5.1 Efectores

5.2 Visores

6. Arquitectura de captura

7. Preparación del PC de datos y del PICO

7.1 PC de datos

7.2 PICO 4 Ultra Enterprise

8. Controles PICO suministrados

9. Diseño de una campaña de datos

9.1 Definir primero la política que se quiere aprender

9.2 Matriz de variación

9.3 Separación train/validation/test

10. Procedimiento de grabación de una sesión

Fase A: congelar configuración

Fase B: seguridad física

Fase C: comprobar flujos sin grabar

Fase D: grabar un episodio

Fase E: tratar fallos y retakes

Fase F: cierre

11. Contrato mínimo del dataset

12. Control de calidad antes de entrenar

12.1 Gates automáticos por episodio

12.2 Revisión humana

12.3 Auditoría de frecuencia

13. Elegir el punto de partida del VLA

13.1 Modelo, fine-tuning y checkpoint no son lo mismo

13.2 Para qué fue hecho el checkpoint actual

13.3 Comparación de las tres estrategias

13.4 A. Continuar checkpoint-40000

13.5 B. Crear un VLA nuevo desde GR00T N1.5

13.6 C. Entrenar desde pesos aleatorios

13.7 Decisiones recomendadas para este proyecto

14. Entrenamiento fuera del robot

15. Validación y despliegue de un candidato

16. Capacidad esperable y límites

Lo que sí habilita la plataforma

Lo que no debe prometerse todavía

17. Piloto recomendado antes de una campaña completa

18. Preguntas abiertas para DSA

19. Fuentes locales y trazabilidad

1. Propósito y alcance

Esta guía define cómo usar el Cruzr S2 real y un PICO 4 Ultra Enterprise para recoger demostraciones reproducibles, convertirlas en un dataset auditable y entrenar o continuar un checkpoint VLA. Integra cuatro fuentes:

- el estado observado del robot tras la actualización v0.2.0;
- el SOP de teleoperación y los instaladores suministrados;
- el paquete VLA GR00T N1.5 con `checkpoint-40000`;
- las pruebas shadow y las experiencias de manipulación realizadas en este robot.

No es una autorización para movimiento VLA autónomo. La activación física sigue condicionada por los bloqueos de [CRUZR S2 VLA SAFE ENABLEMENT.md](#).

Convenciones de evidencia

- **Verificado:** observado en el robot o inspeccionado en los paquetes disponibles.
- **Suministrado:** descrito por el SOP o configuración del proveedor.
- **Recomendado:** criterio de ingeniería para este proyecto, pendiente de validación experimental.
- **Pendiente DSA:** requiere confirmación del proveedor antes de considerarlo soporte oficial.

2. Estado de referencia del sistema

2.1 Robot real

Elemento	Estado de referencia	Evidencia
Plataforma	Cruzr S2	Verificado
Software	imágenes <code>zs2_motion-v0.2.0</code> y <code>zs2_vision-v0.2.0</code>	Verificado
Efector actual	abrazaderas laterales pasivas	Verificado
<code>HW_TYPE</code>	<code>cruzr_s2_v1</code>	Verificado
Dispositivo de teleoperación	<code>TELE_DEVICE=pico</code>	Verificado
Transporte	<code>transmit=local</code>	Verificado
Motion	<code>192.168.11.2</code>	Verificado
Vision/web	<code>192.168.11.3</code>	Verificado
Checkpoint	GR00T N1.5, <code>checkpoint-40000</code>	Instalado y probado en shadow
Ejecución física VLA	deshabilitada	Verificado

El SOP nombra específicamente la compilación `utars-udoke-config-v0.2.0-dac-beta.2.tar.gz`. Las imágenes activas muestran `v0.2.0`, pero ese nombre corto no prueba por sí solo que todos los componentes correspondan exactamente a `dac-beta.2`. La presencia de `pico`, los servicios y los topics demuestra compatibilidad funcional inicial, no equivalencia de build. Esta equivalencia queda como **Pendiente DSA**.

2.2 Interfaces observables relevantes

Teleoperación PICO:

Topic	Tipo
<code>/pico_vr/hand_data</code>	<code>sensor_msgs/msg/JointState</code>
<code>/pico_vr/joy_data</code>	<code>quest_msgs/msg/Joysticks</code>
<code>/pico_vr/pose_data</code>	<code>sensor_msgs/msg/JointState</code>
<code>/pico_vr/tele_data</code>	<code>sensor_msgs/msg/JointState</code>
<code>/mc/teleoperation/enable</code>	<code>std_msgs/msg/Bool</code>
<code>/mc/teleoperation/mode_status</code>	<code>sensor_msgs/msg/JointState</code>
<code>/teleop/enable</code>	<code>std_msgs/msg/Bool</code>

Estado, observación y fuerza del robot:

Topic	Tipo	Uso posible
<code>/sensor/camera/stereo/color/raw</code>	<code>shm_msgs/msg/Image2m</code>	RGB usado por el perfil VLA suministrado
<code>/mc/whole_joint_states</code>	<code>sensor_msgs/msg/JointState</code>	estado articular observado
<code>/mc/sdk/robot_state</code>	<code>mc_state_msgs/msg/RobotState</code>	interfaz prevista, pero sin muestras en pruebas previas
<code>/mc/ft_states/L_hand_ft</code>	<code>geometry_msgs/msg/WrenchStamped</code>	fuerza/par izquierdo para QC y seguridad
<code>/mc/ft_states/R_hand_ft</code>	<code>geometry_msgs/msg/WrenchStamped</code>	fuerza/par derecho para QC y seguridad

El robot dispone además de cámaras RGB-D de chasis y cintura, estéreo, fisheye, profundidad y nubes de puntos. Que existan no significa que el checkpoint actual sepa utilizarlas: el perfil entregado consume una sola imagen RGB. Añadir profundidad o más cámaras exige modificar dataset, configuración y entrenamiento de forma coherente.

3. Qué hace actualmente el VLA suministrado

3.1 Entrada, salida y tareas

El checkpoint recibido utiliza:

- una observación RGB, redimensionada a `224 x 224` por el data config;
- una instrucción de lenguaje;

- 20 posiciones articulares como estado;
- chunks de 10 acciones, cada acción con 20 valores.

Orden exacto de estado y acción:

1. brazo izquierdo: 7 ejes;
2. brazo derecho: 7 ejes;
3. cabeza: 2 ejes;
4. elevador: 3 ejes;
5. cintura: 1 eje.

No contiene acción de chasis, dedos ni pinza eléctrica. Sus cuatro instrucciones son recoger y depositar una caja grande en el nivel inferior o medio.

3.2 Dataset original inspeccionado

El dataset `utars_clamp_and_place_large_box_full_data_bio_lerobot_0319` es LeRobot v2.1 y declara:

- 500 episodios;
- 105 207 frames;
- 500 vídeos;
- aproximadamente 1,9 GB de vídeo;
- 120 FPS declarados;
- imagen RGB `960 x 576`;
- 32 valores de estado almacenados: 20 articulaciones y 12 valores de fuerza/par;
- 20 valores de acción;
- task index, episodio, frame, timestamp e instrucción.

Distribución:

Tarea	Episodios	Frames
recoger caja, nivel inferior	150	34 302
depositar caja, nivel inferior	150	26 411
recoger caja, nivel medio	100	23 861
depositar caja, nivel medio	100	20 633

Hay que auditar los datos antes de reutilizarlos. Por ejemplo, existe un episodio de sólo 9 frames y los metadatos de codec no son completamente consistentes. Los 120 FPS declarados tampoco coinciden con la tasa RGB de unos 12,5 Hz observada en vivo: puede existir remuestreo o repetición de frames. No se debe forzar ningún recorder a 120 FPS sin entender primero la exportación.

Aunque las fuerzas están almacenadas, `Utars_1RGBDataConfig` selecciona únicamente las 20 articulaciones como entrada al modelo. En consecuencia, el checkpoint actual **no está condicionado por fuerza**. Las fuerzas siguen siendo valiosas para seguridad, detección de contacto y control de calidad.

3.3 Resultado shadow

El modelo se cargó y produjo chunks finitos `10 x 20`. Una prueba de tarea generó dos chunks en aproximadamente diez segundos; la primera inferencia fue más lenta que las siguientes. No hubo publicadores en `/mc/sdk/robot_command`.

Desde `home`, ocho ejes de brazo excedieron el límite conservador y la diferencia llegó a aproximadamente 1,35 rad. El validador rechazó el movimiento. Por tanto, el modelo depende de una postura inicial de preparación que no se debe sustituir por `home`.

4. VLA frente a programación tradicional

Necesidad	Enfoque preferente
caja conocida, geometría y alturas controladas	detector <code>workbin</code> + secuencia determinista
pequeñas variaciones medibles de pose	detector + servo visual/AprilTag + trayectoria parametrizada
objetos, posiciones y escenas variables dentro de una misma familia	ajuste VLA con demostraciones variadas
requisito de repetibilidad, trazabilidad y validación industrial	programación tradicional siempre que sea suficiente
manipulación con dedos, acción distinta a 20D	nuevo perfil de datos y salida; el checkpoint actual no basta

El detector `workbin` y las tareas BYD no son un VLA: localizan la caja y ejecutan primitivas programadas, usando geometría y realimentación de fuerza/contacto. Esto explica por qué el robot puede adaptarse a la posición de una caja dentro de un rango sin aprendizaje generalista.

5. Matriz de compatibilidad

5.1 Efectores

Efector	HW_TYPE	Teleoperación	Checkpoint actual
abrazaderas	<code>cruZR_s2_v1</code>	soportada	compatible por diseño
manos v3/v4	<code>cruZR_s2_v1_sps</code>	soportadas por el stack, pendiente validar captura	incompatible con la salida 20D
pinza de dos dedos/cilindro	<code>cruZR_s2_v1_gripper</code>	descrita en SOP	incompatible sin agregar su acción

No se deben mezclar episodios de diferentes efectores en el mismo perfil sin registrar el efector y definir una representación de acción común. Cambiar `HW_TYPE` requiere volver a desplegar/recrear los componentes de motion según el SOP; no basta editar una variable dentro de un contenedor en ejecución.

5.2 Visores

Visor	Estado de soporte comunicado
PICO 4 Ultra Enterprise	soportado
PICO 4 Ultra de consumo	no soportado oficialmente; adaptación propia
Meta Quest 3	citado en el SOP

Visor	Estado de soporte comunicado
Oculus/Meta Quest 2	no confirmado

El proyecto debe utilizar el **PICO 4 Ultra Enterprise ya disponible** y no asumir equivalencia de firmware o permisos con la versión de consumo.

6. Arquitectura de captura

```

PICO 4 Ultra Enterprise
├─ XRoboToolkit-PICO (poses, mandos y opcionalmente trackers)
│   └─ USB / red compartida
│       └─ PC de datos Ubuntu
│           ├── XRoboToolkit PC Service
│           ├── ubt-controller 5.3.0 (PICO, 90 Hz configurados)
│           ├── ubt-remote-control 4.1.0
│           └─ centro de captura/exportación
│               ├── observación RGB
│               ├── estado articular
│               ├── acción/objetivo teleoperado
│               ├── fuerza/par y timestamps
│               └─ metadatos de tarea y episodio
│                   └─ LeRobot v2.1 auditable
│                       └─ entrenamiento GPU fuera del robot
│                           └─ checkpoint candidato
│                               └─ offline → shadow → físico limitado

```

El servicio XR incluye un ejemplo `RobotDataRecorder` que guarda datos XR brutos (`head.txt`, `hand.txt`, `controller.txt`, `body.csv` y `motion.txt`) y timestamps. Es útil para diagnóstico y archivo, pero no reemplaza la captura sincronizada de imagen, estado y acción del robot ni produce por sí solo el dataset LeRobot esperado por GROOT.

7. Preparación del PC de datos y del PICO

7.1 PC de datos

Requisitos suministrados:

- Ubuntu 22.04 o posterior;
- Chrome actualizado;
- Ethernet directa al robot;
- IP estática del PC `192.168.11.250` para el enlace indicado en el SOP;
- ADB;
- una sola variante de XRoboToolkit PC Service;
- `ubt-controller` 5.3.0;
- `ubt-remote-control` 4.1.0.

Comprobaciones no destructivas recomendadas:

```
dpkg -l | grep -E 'ubt-controller|ubt-remote-control|xrobotoolkit'
systemctl status ubt-controller.service --no-pager
ip -br address
adb devices
```

El archivo de configuración suministrado está en:

```
/opt/ubt/ubt_controller/config/config.json
```

Para operación local debe conservar coherencia entre:

```
{
  "transmit": "local",
  "signal_server_url": "ws://192.168.11.3:4000",
  "push_rate": 90,
  "control_device": "pico",
  "enable_adb_reverse": 1
}
```

Después de un cambio autorizado:

```
sudo systemctl restart ubt-controller.service
sudo systemctl status ubt-controller.service --no-pager
```

No se deben copiar contraseñas, tokens o datos personales al manifiesto de sesión.

7.2 PICO 4 Ultra Enterprise

1. Actualizar el sistema del visor.
2. Activar el modo desarrollador pulsando diez veces la versión de software, según el SOP.
3. Desactivar el apagado automático de pantalla y suspensión.
4. Instalar por USB:

```
adb install -g XRoboToolkit-PICO-1.1.1.apk
```

5. Durante la teleoperación, mantener el Wi-Fi del PICO apagado para evitar interferencias, tal como indica el SOP.
6. Conectar USB y elegir `Shared network (connect USB first)` en XRoboToolkit.
7. Conectar al servicio del PC y comprobar estado verde `working`.
8. Elegir `Head + Controller` y `Send`.
9. Si se usan trackers, seleccionar el modo correcto y calibrarlos en **cada sesión**. No modificar sus variables durante una teleoperación activa.
10. Minimizar la aplicación para evitar pulsaciones accidentales.

Con trackers, el SOP contempla configuraciones de tres o cinco unidades. El tracker de cintura se coloca detrás y no debe quedar cubierto. Registrar en el manifiesto el número, firmware, colocación y resultado de calibración.

8. Controles PICO suministrados

Control	Función
Y	iniciar/detener teleoperación
X	alternar modo de cuerpo completo en sitio y modo móvil
joystick izquierdo, en sitio	giro/inclinación de cintura
clic joystick izquierdo	activar/desactivar protección de fuerza del brazo izquierdo; por defecto activa
joystick derecho, en sitio	subir/bajar elevador
clic joystick derecho	activar/desactivar protección de fuerza del brazo derecho; por defecto activa
joystick izquierdo, móvil	avance/retroceso de base
joystick derecho, móvil	giro de base
B	iniciar/detener captura de un episodio
A	reset de tren superior, sólo en modo en sitio; no resetea elevador
trigger izquierdo/derecho	cerrar mano mientras se mantiene; abrir al soltar
grip izquierdo/derecho	ese brazo y cintura siguen al operador, sólo en modo en sitio

Antes de terminar, volver siempre al modo de cuerpo completo en sitio. Si Y o B no responde, el SOP menciona desconectar USB como recurso de emergencia, pero sólo después de asegurar robot, carga y personas. No debe convertirse en un procedimiento normal.

9. Diseño de una campaña de datos

9.1 Definir primero la política que se quiere aprender

Cada tarea debe tener:

- una instrucción breve, exacta y estable en inglés;
- estado inicial permitido;
- condición observable de éxito;
- condición de aborto;
- efector y carga;
- rango de alturas, offsets, yaw y tamaños;
- modalidades y acción con orden y unidades documentados.

Ejemplo para extender el checkpoint de abrazaderas:

```
Task ID: 4
Instruction: Pick up the blue workbin from the table.
Initial state: robot at the marked pre-pick zone; arms in the official ready pose.
Success: workbin is stably suspended for 1 second without slip.
Abort: loss of detection, collision, excessive force, unstable clamp or E-stop.
```

No cambie sin necesidad entre sinónimos como `pick`, `grab` y `take`. La consistencia lingüística reduce una fuente de variabilidad que no aporta habilidad motora.

9.2 Matriz de variación

Para que el modelo se adapte, las demostraciones deben cubrir deliberadamente:

- offset lateral y distancia;
- yaw del objeto;
- altura de mesa/estante;
- tamaño, color, textura y masa dentro del rango autorizado;
- iluminación y fondo;
- presencia de distractores seguros;
- aproximación desde distintas poses válidas;
- operadores distintos, si se quiere reducir sesgo de estilo.

No varíe todo a la vez en el piloto. Empiece con una cuadrícula pequeña que permita identificar qué condición provoca errores. La recomendación inicial es 10 episodios exitosos por celda de un piloto y, tras validar el pipeline, 50–100 demostraciones exitosas por variante de tarea. El dataset del proveedor usa 100–150 episodios por tarea; esto sirve como referencia, no como garantía.

9.3 Separación train/validation/test

Dividir por **sesión, escena, objeto y condición**, no por frames aleatorios del mismo vídeo. Si frames casi idénticos aparecen en train y test, el resultado parecerá mejor sin medir generalización.

Una propuesta inicial:

- 70 % entrenamiento;
- 15 % validación;
- 15 % test bloqueado;
- al menos un objeto, una posición o una sesión completa no vistos en train.

10. Procedimiento de grabación de una sesión

Usar también el [checklist imprimible](#).

Fase A: congelar configuración

1. Crear un ID de sesión y completar `session_manifest.example.yaml`.
2. Registrar versión del robot, hashes de configuración, `HW_TYPE`, `TELE_DEVICE`, `transmit`, efector, carga y operador.
3. Registrar mapa/escena, cámaras, resolución y topics.
4. Confirmar que no hay cambios de software pendientes durante la sesión.
5. Reservar espacio: el dataset suministrado equivale aproximadamente a 7,8 GB por hora de vídeo. Mantener al menos tres veces la estimación por archivos temporales, XR bruto y copias de seguridad.

Fase B: seguridad física

1. Cargador desconectado.
2. Efector fijado, cableado y correspondiente al `HW_TYPE`.

3. Teleoperador fuera del alcance del robot.
4. Observador independiente con paro físico preparado.
5. Zona completa y recorrido de base despejados.
6. Protecciones de fuerza de ambos brazos activas.
7. Carga y masa dentro del rango autorizado.
8. Regla de aborto acordada verbalmente.

Fase C: comprobar flujos sin grabar

1. Arrancar el robot con el procedimiento validado para el hardware real.
2. Verificar estado, batería, paros y cargador.
3. Comprobar conexión Ethernet robot-PC.
4. Conectar PICO por USB, estado XR verde y Wi-Fi del visor apagado.
5. Recalibrar trackers si se usan.
6. Comprobar movimiento pequeño de cada elemento necesario sin objeto.
7. Volver a la postura inicial canónica.
8. Confirmar que imagen, estado, acción y fuerzas están llegando y que los timestamps avanzan.

Fase D: grabar un episodio

1. Colocar el objeto según la celda de la matriz y registrar su pose real.
2. Llevar el robot a la postura inicial canónica **fuera de la grabación**.
3. Confirmar instrucción/task ID y número de episodio.
4. Esperar aproximadamente un segundo con la escena estable.
5. Pulsar **B** justo antes del primer movimiento significativo.
6. Ejecutar una demostración fluida, intencional y sin correcciones nerviosas.
7. Al alcanzar el éxito, mantener el resultado estable aproximadamente un segundo para que el final sea observable.
8. Pulsar **B** para cerrar el episodio.
9. Volver a postura segura y preparar el siguiente episodio fuera de captura.
10. Clasificar inmediatamente el episodio como aceptado, rechazado o pendiente en [episode_log.csv](#).

Evitar pausas largas, conversaciones, reposicionamientos manuales o resets dentro del episodio. No concatenar dos intentos bajo un mismo episodio.

Fase E: tratar fallos y retakes

El cargador actual no dispone de un campo de éxito que el entrenamiento use de forma explícita. Por ello:

- un intento fallido se conserva en un área de cuarentena para análisis;
- no se mezcla con las demostraciones de behavioral cloning salvo que se cambie conscientemente el método de entrenamiento;
- un retake obtiene un nuevo episode ID;
- nunca se sobrescribe el original sin dejar trazabilidad;
- anotar colisión, saturación, pérdida visual, acción incorrecta o error humano.

Fase F: cierre

1. Finalizar teleoperación en modo de cuerpo completo en sitio.
2. Detener captura y verificar que no queda un episodio abierto.

3. Realizar apagado normal; el paro de emergencia no es el procedimiento ordinario de apagado.
4. Generar inventario y hashes SHA-256.
5. Copiar a almacenamiento secundario antes de borrar temporales.
6. Ejecutar QC automático y revisar una muestra visual el mismo día.
7. Firmar el manifiesto con incidencias y episodios aceptados/rechazados.

11. Contrato mínimo del dataset

Para continuar el checkpoint actual sin cambiar el espacio de acción:

Campo	Requisito
formato	LeRobot v2.1 o conversión demostrablemente equivalente
<code>video.rgb</code>	RGB legible y sincronizado; documentar codec real
estado	20 articulaciones en el orden exacto del checkpoint
acción	20 objetivos articulares, mismo orden y unidades
fuerza/par	12 valores opcionales almacenados para QC; no consumidos por el perfil actual
timestamp	monotónico y relacionado con el reloj fuente
task	ID e instrucción estables
episodio	frontera inequívoca y un solo intento
metadatos	robot, versión, efector, escena, operador y resultado

La acción de cada frame no son diez archivos independientes: el data loader forma una ventana de 10 acciones consecutivas. La continuidad temporal es obligatoria.

La variante `Utars_1RGB_From_TeleopDataConfig` encontrada no coincide con el perfil 20D: usa dimensiones heredadas de 17/16 y omite elementos. No debe usarse para este robot/checkpoint sin aclaración del proveedor o una corrección versionada y probada.

12. Control de calidad antes de entrenar

12.1 Gates automáticos por episodio

- vídeo abre, decodifica y tiene duración coherente;
- no hay NaN, infinito ni arrays de dimensión incorrecta;
- nombres, orden y unidades articulares coinciden con el contrato;
- timestamps son monótonos y no tienen saltos inexplicados;
- estado, acción e imagen están alineados dentro de un frame de la línea temporal elegida;
- frames perdidos o repetidos se cuantifican;
- no hay saltos articulares imposibles ni velocidades fuera del límite;
- fuerzas no muestran impacto o saturación no anotados;
- task ID e instrucción existen en el catálogo;
- inicio y final cumplen la definición de la tarea;

- el episodio no tiene 0–9 frames por un error de captura.

Los umbrales finales deben derivarse de tasas reales y límites del robot. No se deben copiar umbrales genéricos sin medir el pipeline.

12.2 Revisión humana

Reproducir como mínimo:

- todos los episodios marcados con incidencia;
- el primero y último de cada sesión;
- una muestra aleatoria por tarea/condición;
- todos los outliers de duración, fuerza, velocidad o pérdida de frames.

Comprobar que la vista disponible para el modelo permite inferir el objeto y el resultado. Si el teleoperador ve algo que la cámara del modelo no ve, la demostración puede ser imposible de aprender.

12.3 Auditoría de frecuencia

Registrar por separado:

- tasa de envío XR (`push_rate`, actualmente 90);
- tasa real de imagen;
- tasa de estado y acción;
- FPS declarado del dataset;
- cantidad de frames RGB únicos.

No interpretar `120 FPS` como 120 imágenes nuevas por segundo hasta verificar los archivos. Si se remuestrea, documentar algoritmo, reloj maestro y política de interpolación/retención.

13. Elegir el punto de partida del VLA

13.1 Modelo, fine-tuning y checkpoint no son lo mismo

Un **checkpoint** es una instantánea de los pesos y la configuración de un modelo en un momento del entrenamiento. Tanto un modelo continuado como uno entrenado desde cero acabarán guardándose en checkpoints. Por tanto, la decisión real no es «usar o no un checkpoint», sino elegir de qué pesos partir:

1. continuar el checkpoint de cajas entregado por DSA;
2. partir del modelo fundacional preentrenado GR00T N1.5, sin heredar el ajuste de cajas de DSA;
3. partir de pesos aleatorios y entrenar arquitectura, visión, lenguaje y acción desde cero.

La segunda opción es normalmente lo que se quiere decir en este proyecto por «crear un VLA nuevo». No reutiliza `checkpoint-40000`, pero sí aprovecha el conocimiento visual y lingüístico del modelo fundacional.

13.2 Para qué fue hecho el checkpoint actual

El `checkpoint-40000` suministrado está especializado en:

- Cruzr S2 con abrazaderas laterales pasivas;
- una sola cámara RGB;
- estado y acción articulares de 20 dimensiones;
- aproximaciones que parten de una postura VLA-ready compatible;

- recoger y depositar una caja grande;
- niveles inferior y medio de la escena de entrenamiento;
- cuatro instrucciones en inglés correspondientes a esas combinaciones.

No fue hecho para:

- controlar el chasis como parte de la salida aprendida;
- accionar manos, dedos o una pinza eléctrica;
- consumir fuerza/par como entrada, aunque esos datos existan en el dataset;
- utilizar profundidad, point cloud o varias cámaras;
- cualquier caja, estante o altura sin límite;
- tareas generales como botellas, herramientas, botones o ensamblaje.

Puede generalizar moderadamente dentro de la distribución cubierta por sus demostraciones, pero no debe presentarse como un manipulador universal.

13.3 Comparación de las tres estrategias

Estrategia	Punto de partida	Recomendada cuando	Coste y riesgo
continuar DSA	checkpoint-40000	misma cámara, abrazaderas, acción 20D y tareas de cajas relacionadas	menor cantidad de datos y convergencia rápida; riesgo de olvidar tareas antiguas
nuevo VLA de dominio	nvidia/GR00T-N1.5-3B	nuevo efector, nueva salida, modalidades o tareas alejadas del demo de cajas	más datos y validación; conserva conocimiento fundacional
desde cero real	pesos aleatorios	arquitectura incompatible, restricción legal/licencia o dominio sin un preentrenado aprovechable	dataset multimodal y cómputo enormes; mayor riesgo, casi nunca recomendado aquí

13.4 A. Continuar checkpoint-40000

Es el camino preferente cuando se mantienen:

- abrazaderas;
- una cámara RGB;
- orden 20D de estado/acción;
- familia de tareas de cajas;
- cinemática y postura inicial compatibles.

Ejemplos adecuados:

- más tamaños o colores de cajas dentro de la capacidad mecánica;
- nuevas posiciones laterales o yaw;
- otras alturas alcanzables con las mismas abrazaderas;
- variaciones de iluminación y entorno industrial;
- transferencia de caja entre mesas una vez separada la navegación de la política de manipulación.

Procedimiento:

1. conservar una copia inmutable de checkpoint-40000;
2. grabar episodios nuevos con exactamente el mismo contrato 1 RGB/20D;
3. mezclar demostraciones antiguas y nuevas;

4. continuar el fine-tuning con tasa de aprendizaje conservadora;
5. evaluar tanto las tareas nuevas como las cuatro antiguas;
6. rechazar el candidato si mejora lo nuevo pero degrada lo anterior por debajo del criterio acordado.

Continuar únicamente con datos nuevos puede provocar olvido catastrófico.

13.5 B. Crear un VLA nuevo desde GR00T N1.5

Es la recomendación para una política que **no dependa del ajuste previo de cajas**, pero aproveche el modelo fundacional. Debe elegirse al introducir:

- manos v4 con articulaciones de dedos;
- pinza accionada o un nuevo efector;
- otra dimensión u orden de acción;
- acciones de chasis dentro de la política;
- varias cámaras, profundidad u otras observaciones;
- fuerza/par como entrada real del modelo;
- tareas alejadas: botellas, herramientas, picking de piezas, pulsadores, clasificación o ensamblaje.

Procedimiento:

1. definir el nuevo contrato de observación, estado y acción antes de grabar;
2. crear un nuevo `DataConfig` con nombres, orden, unidades, normalización y horizonte explícitos;
3. instrumentar el recorder para exportar exactamente ese contrato;
4. grabar un piloto y validar extremo a extremo el loader;
5. ampliar un dataset equilibrado, con train/validation/test separados por sesión y condición;
6. inicializar desde `nvidia/GR00T-N1.5-3B`, no desde `checkpoint-40000`;
7. entrenar un nuevo directorio de experimento y no sobrescribir pesos DSA;
8. validar offline, en shadow y físicamente mediante la progresión de la sección 15.

Si el nuevo espacio incluye dedos, el dataset debe registrar su estado y su comando; no es posible aprenderlos a partir de episodios 20D que nunca los contuvieron.

13.6 C. Entrenar desde pesos aleatorios

Entrenar «desde cero real» implica no utilizar ni `checkpoint-40000` ni GR00T N1.5 preentrenado. Habría que aprender representación visual, comprensión de lenguaje y control a partir de los datos propios.

Sólo tiene sentido si:

- la arquitectura necesaria es incompatible con GR00T;
- una restricción de licencia o propiedad intelectual prohíbe pesos externos;
- se dispone de un corpus multimodal masivo y cómputo suficiente;
- existe un equipo capaz de entrenar y mantener un foundation model;
- las alternativas preentrenadas han sido evaluadas y fallan por razones demostrables.

No se recomienda para el Cruzr S2 actual. Las demostraciones que se pueden recoger con un solo robot son apropiadas para fine-tuning, pero no suelen ser suficientes para crear desde cero la capacidad visual-lingüística de un VLA.

13.7 Decisiones recomendadas para este proyecto

Caso	Decisión recomendada
cajas nuevas con abrazaderas y 1 RGB	continuar <code>checkpoint-40000</code> mezclando datos antiguos
depositar cajas a más alturas con las mismas abrazaderas	continuar el checkpoint si la cinemática y la vista siguen siendo válidas
manos v4 y manipulación con dedos	nuevo DataConfig y nuevo fine-tuning desde GR00T N1.5
botella con pinza de dos dedos	nuevo perfil desde GR00T N1.5; detector tradicional puede ser mejor si el entorno es fijo
fuerza como entrada aprendida	nuevo perfil/modelo desde GR00T N1.5 y dataset con fuerza sincronizada
navegación entre mesas	mantener navegación LiDAR tradicional; usar VLA sólo para la manipulación, salvo proyecto de investigación específico
investigación de una arquitectura completamente propia	desde cero sólo con presupuesto, datos y justificación extraordinarios

14. Entrenamiento fuera del robot

El script suministrado parte de `nvidia/GR00T-N1.5-3B` y contempla ajuste de proyector y difusión, LoRA opcional, varios datasets y checkpoints periódicos. El checkpoint recibido alcanzó `global_step=40000`, aproximadamente 6,08 épocas, con batch 16. No registra un mejor checkpoint por evaluación; `best_model_checkpoint` es nulo. La pérdida de entrenamiento no sustituye una evaluación retenida.

Flujo recomendado:

1. Congelar dataset, hashes y data config.
2. Validar conversión con 2-5 episodios.
3. Ejecutar overfit controlado sobre un subconjunto para probar el pipeline.
4. Entrenar en una estación NVIDIA/CUDA, no en el robot de producción.
5. Guardar argumentos, commit, versiones, seed, logs y checkpoints.
6. Evaluar cada checkpoint sobre splits retenidos y simulación/offline.
7. Elegir por métrica de validación y pruebas de rollout, no por el número de step más alto.

El requisito exacto de GPU, VRAM y tiempo no está documentado en el material inspeccionado; debe medirse o confirmarse con DSA antes de dimensionar compras.

15. Validación y despliegue de un candidato

Secuencia obligatoria:

1. **Offline:** formas, NaN, rangos, continuidad, latencia y replay de episodios.
2. **Dataset retenido:** éxito por tarea y por condición no vista.
3. **Shadow en robot:** inferencia con estado e imagen reales, sin publicador de comandos.
4. **Postura inicial oficial:** comparar primer chunk con la postura real.
5. **Ejecutor S2 seguro:** mapping explícito, límites, rate limit, timeout, deadman, estado fresco y paro.
6. **Movimiento sin objeto:** velocidad y amplitud reducidas.

7. **Objeto ligero/controlado:** zona cerrada y observador con paro.
8. **Variaciones progresivas:** sólo después de superar el nivel anterior.
9. **Rollback:** checkpoint anterior y servicios VLA detenidos disponibles.

Para el checkpoint suministrado sigue pendiente resolver la primitiva de preparación `clamp_s2_joints_trajectory` y disponer de un ejecutor 20D seguro. No activar el ejecutor SDK incompleto ni usar el perfil Walker de 30 DOF.

16. Capacidad esperable y límites

Lo que sí habilita la plataforma

- teleoperación de brazos, cintura, elevador, cabeza y base según modo;
- captura de episodios mediante `B`;
- control de manos si el hardware/configuración correspondiente está activo;
- registro de XR, imagen, articulaciones y fuerza si el centro de captura los exporta correctamente;
- replay determinista de demostraciones;
- fine-tuning VLA para adaptación estadística a variaciones cubiertas por datos;
- inferencia GR00T N1.5 a bordo, ya demostrada en shadow.

Lo que no debe prometerse todavía

- aprendizaje automático por una sola demostración;
- manipulación con dedos usando el checkpoint de abrazaderas;
- generalización a objetos o alturas ausentes del dataset;
- uso de fuerza por el modelo actual;
- navegación VLA del chasis con la salida 20D;
- que cualquier grabación XR sea directamente entrenable;
- movimiento físico del checkpoint antes de cerrar los gates de seguridad.

17. Piloto recomendado antes de una campaña completa

1. Mantener abrazaderas, `HW_TYPE=cruzer_s2_v1`, `pico` y `local`.
2. Definir una sola tarea sencilla y una sola altura.
3. Grabar 10 episodios exitosos y 2 fallidos en cuarentena.
4. Exportar y comprobar video, 20D estado, 20D acción, fuerza, task y timestamps.
5. Reproducir visualmente los 10 episodios.
6. Convertir a LeRobot y ejecutar todos los gates de QC.
7. Probar que el loader `Utars_1RGBDataConfig` carga el lote.
8. Hacer un overfit del subconjunto fuera del robot.
9. Ejecutar inferencia offline y luego shadow.
10. Sólo entonces ampliar posiciones, objetos y episodios.

18. Preguntas abiertas para DSA

1. ¿El sistema instalado corresponde exactamente a `v0.2.0-dac-beta.2`, aunque las imágenes se llamen `v0.2.0`?
2. ¿Qué aplicación/servicio produce el dataset LeRobot y dónde lo exporta?
3. ¿Cuál es el reloj maestro y cómo se sincronizan 90 Hz XR, RGB y 120 FPS declarados?
4. ¿Qué significado exacto tiene cada campo 20D y en qué unidades?
5. ¿Cómo se marca oficialmente el éxito o rechazo de un episodio?
6. ¿Se puede capturar fuerza/par como entrada de entrenamiento soportada?
7. ¿Cuál es el data config oficial para manos v4 y cuál es su acción?
8. ¿Cuál es la postura oficial VLA ready y dónde se define `clamp_s2_joints_trajectory`?
9. ¿Qué ejecutor 20D y límites recomienda DSA para Cruzr S2?
10. ¿Qué GPU/VRAM y versión exacta de GR00T recomiendan para continuar `checkpoint-40000`?

19. Fuentes locales y trazabilidad

- Índice del paquete PICO/UTATS: [utats/README.md](#).
- SOP suministrado: PDF local dentro de `utats/`.
- Paquete VLA local ignorado: `cruzrss2_vla_pack-002/`.
- Config de datos: `cruzrss2_vla_pack-002/data_config.py`.
- Entrenamiento: `cruzrss2_vla_pack-002/gr00t_finetune.py`.
- Dataset de referencia: `cruzrss2_vla_pack-002/data/utars_clamp_and_place_large_box_full_data_bio_lerobot_0319/`.
- Checkpoint: `cruzrss2_vla_pack-002/weight/checkpoint-40000/`.
- Scripts shadow: [scripts/vla/](#).

Los paquetes grandes no se versionan en Git. Toda campaña debe registrar sus hashes y conservar una copia inmutable de dataset, data config, código y checkpoint.